Интеллектуальная платформа управления задачами

Детальная архитектура системы — Техническое задание для ИИ-разработчика

> **Стек:** FastAPI · SQLAlchemy · PostgreSQL · Qdrant · LangGraph · LangChain · React · TypeScript

Содержание

1. Обзор системы
2. Модель данных PostgreSQL
3. Backend: файловая структура
4. Backend: core-модули
5. Backend: SQLAlchemy-модели
6. Backend: Pydantic-схемы
7. Backend: API-роутеры (все эндпоинты)
8. Backend: сервисный слой
9. Агентный слой (LangGraph)
10. Frontend: файловая структура
11. Frontend: API-клиент
12. Frontend: AuthContext и управление токенами
13. Frontend: роутинг и защищённые маршруты
14. Frontend: страницы и компоненты
15. Переменные окружения
16. Docker Compose

1. Обзор системы

Роли пользователей

Роль	Константа	Возможности
Администратор	`ADMIN`	Управление пользователями, настройка custom rules, просмотр всего
Аналитик	`ANALYST`	Создание/редактирование задач, запуск валидации, управление предложениями
Разработчик	`DEVELOPER`	Чтение задач, участие в чате

Роль	Константа	Возможности
Тестировщик	`TESTER`	Чтение задач, участие в чате
Менеджер	`MANAGER`	Просмотр всех задач и аналитики проекта

Стратегия JWT

- access_token — короткоживущий (15 мин), stateless, хранится в памяти (JS variable)
- refresh_token — долгоживущий (7 дней), хранится в `httpOnly` cookie; хэш — в таблице `refresh_tokens` в БД (для ревокации)
- **Ротация** refresh_token — при каждом `/auth/refresh` старый токен инвалидируется, выдаётся новый

Ключевые принципы безопасности

- Пароли — bcrypt (rounds=12)
- CORS — только allowed origins из `.env`
- refresh_token в `httpOnly; Secure; SameSite=Lax` cookie (недоступен JS)
- access_token в памяти (не в `localStorage`) → защита от XSS
- При краже refresh_token: повторное использование инвалидированного токена → немедленная ревокация всей семьи токенов пользователя

2. Модель данных PostgreSQL

Полный DDL

```
-- Расширения
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ═══════════════════════════════════════════
-- ENUM типы
-- ═══════════════════════════════════════════
CREATE TYPE user_role AS ENUM ('ADMIN', 'ANALYST', 'DEVELOPER', 'TESTER', 'MANAGER');

CREATE TYPE task_status AS ENUM (
    'draft',          -- создана, не отправлена на валидацию
    'validating',     -- агент валидирует
    'needs_rework',   -- вернули на доработку
    'ready_for_dev',  -- прошла валидацию
    'in_progress',    -- в работе
    'done'            -- завершена
);

CREATE TYPE message_type AS ENUM (
    'general',            -- обычное обсуждение (агент не отвечает)
    'question',           -- вопрос по требованиям
    'change_proposal',    -- предложение изменить требование
    'agent_answer',       -- ответ QA Agent
```

```
    'agent_proposal'      -- ответ ChangeTracker Agent
);

CREATE TYPE proposal_status AS ENUM ('new', 'accepted', 'rejected');

-- ────────────────────────────────────────────
-- Пользователи
-- ────────────────────────────────────────────
CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email           VARCHAR(255) NOT NULL UNIQUE,
    password_hash   TEXT NOT NULL,
    full_name       VARCHAR(255) NOT NULL,
    role            user_role NOT NULL DEFAULT 'DEVELOPER',
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);

-- ────────────────────────────────────────────
-- Refresh-токены (для ревокации)
-- ────────────────────────────────────────────
CREATE TABLE refresh_tokens (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_hash      TEXT NOT NULL UNIQUE,      -- SHA-256 от raw токена
    family_id       UUID NOT NULL,             -- группа токенов одной сессии
    expires_at      TIMESTAMPTZ NOT NULL,
    revoked         BOOLEAN NOT NULL DEFAULT FALSE,
    revoked_at      TIMESTAMPTZ,
    user_agent      TEXT,                      -- для отображения активных сессий
    ip_address      INET,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_refresh_tokens_user_id ON refresh_tokens(user_id);
CREATE INDEX idx_refresh_tokens_token_hash ON refresh_tokens(token_hash);
CREATE INDEX idx_refresh_tokens_family_id ON refresh_tokens(family_id);

-- ────────────────────────────────────────────
-- Проекты
-- ────────────────────────────────────────────
CREATE TABLE projects (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name            VARCHAR(255) NOT NULL,
    description     TEXT,
    created_by      UUID NOT NULL REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Участники проекта (роль на уровне проекта может отличаться от глобальной)
CREATE TABLE project_members (
    project_id      UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
```

```
    user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role            user_role NOT NULL,
    joined_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (project_id, user_id)
);

-- ────────────────────────────────────────────
-- Задачи
-- ────────────────────────────────────────────
CREATE TABLE tasks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id      UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    title           VARCHAR(500) NOT NULL,
    content         TEXT NOT NULL DEFAULT '',   -- основной текст требования
    tags            TEXT[] NOT NULL DEFAULT '{}',
    status          task_status NOT NULL DEFAULT 'draft',
    created_by      UUID NOT NULL REFERENCES users(id),
    assigned_to     UUID REFERENCES users(id),
    validation_result  JSONB,   -- {verdict, issues[], questions[], validated_at}
    indexed_at      TIMESTAMPTZ,              -- когда проиндексировано в Qdrant
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_tasks_project_id ON tasks(project_id);
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_tags ON tasks USING gin(tags);

-- Вложения к задаче
CREATE TABLE task_attachments (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    task_id         UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    filename        VARCHAR(500) NOT NULL,
    content_type    VARCHAR(100) NOT NULL,
    storage_path    TEXT NOT NULL,            -- путь в S3 / local storage
    alt_text        TEXT,                     -- заполняется после Vision LLM
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ────────────────────────────────────────────
-- Сообщения в чате
-- ────────────────────────────────────────────
CREATE TABLE messages (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    task_id         UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    author_id       UUID REFERENCES users(id),   -- NULL если автор — агент
    agent_name      VARCHAR(50),                 -- 'ManagerAgent', 'QAAgent', 'ChangeTrac
    message_type    message_type NOT NULL DEFAULT 'general',
    content         TEXT NOT NULL,
    source_ref      JSONB,     -- {task_id, chunk_ids[], collection} — источник ответа аге
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_messages_task_id ON messages(task_id);
CREATE INDEX idx_messages_created_at ON messages(task_id, created_at);
```

```
-- ────────────────────────────────────────────
-- Предложения изменений
-- ────────────────────────────────────────────
CREATE TABLE change_proposals (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    task_id         UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    source_message_id UUID REFERENCES messages(id),
    proposed_by     UUID REFERENCES users(id),
    proposal_text   TEXT NOT NULL,
    status          proposal_status NOT NULL DEFAULT 'new',
    reviewed_by     UUID REFERENCES users(id),
    reviewed_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_proposals_task_id ON change_proposals(task_id);
CREATE INDEX idx_proposals_status ON change_proposals(status);

-- ────────────────────────────────────────────
-- Пользовательские правила валидации
-- ────────────────────────────────────────────
CREATE TABLE custom_rules (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id      UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    title           VARCHAR(255) NOT NULL,
    description     TEXT NOT NULL,            -- текст правила для LLM
    applies_to_tags TEXT[] NOT NULL DEFAULT '{}',  -- пустой массив = применять ко всем
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    created_by      UUID NOT NULL REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ────────────────────────────────────────────
-- Тригер: автообновление updated_at
-- ────────────────────────────────────────────
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_users_updated_at       BEFORE UPDATE ON users       FOR EACH ROW EXEC
CREATE TRIGGER trg_projects_updated_at    BEFORE UPDATE ON projects    FOR EACH ROW EXE
CREATE TRIGGER trg_tasks_updated_at       BEFORE UPDATE ON tasks       FOR EACH ROW EXEC
```

3. Backend: файловая структура

```
backend/
├── main.py                 # точка входа FastAPI, подключение роутеров
├── pyproject.toml
├── .env
│
├── app/
│   ├── core/
│   │   ├── config.py       # Settings через pydantic-settings
```

```
│   │   ├── security.py       # JWT, bcrypt, token utils
│   │   ├── dependencies.py   # get_current_user, require_role, get_db
│   │   └── database.py       # AsyncEngine, AsyncSession, Base
│   │
│   ├── models/               # SQLAlchemy ORM-модели (один файл = одна таблица)
│   │   ├── user.py
│   │   ├── refresh_token.py
│   │   ├── project.py
│   │   ├── task.py
│   │   ├── message.py
│   │   ├── change_proposal.py
│   │   └── custom_rule.py
│   │
│   ├── schemas/              # Pydantic v2 схемы (request / response)
│   │   ├── auth.py
│   │   ├── user.py
│   │   ├── project.py
│   │   ├── task.py
│   │   ├── message.py
│   │   └── proposal.py
│   │
│   ├── routers/              # FastAPI APIRouter
│   │   ├── auth.py           # /auth/*
│   │   ├── users.py          # /users/*  (admin)
│   │   ├── projects.py       # /projects/*
│   │   ├── tasks.py          # /projects/{id}/tasks/*
│   │   ├── validation.py     # /tasks/{id}/validate
│   │   ├── chat.py           # /tasks/{id}/chat
│   │   └── proposals.py      # /tasks/{id}/proposals
│   │
│   ├── services/             # Бизнес-логика (без HTTP-контекста)
│   │   ├── auth_service.py
│   │   ├── task_service.py
│   │   ├── chat_service.py
│   │   ├── rag_service.py
│   │   └── proposal_service.py
│   │
│   └── agents/               # LangGraph графы
│       ├── state.py           # TypedDict для всех State
│       ├── validation_graph.py
│       ├── chat_graph.py
│       └── rag_pipeline.py
```

4. Backend: core-модули

`app/core/config.py`

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    # Database
    DATABASE_URL: str              # postgresql+asyncpg://user:pass@host/db
```

```
    # JWT
    JWT_SECRET_KEY: str             # 64+ random bytes, base64
    JWT_ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 15
    REFRESH_TOKEN_EXPIRE_DAYS: int = 7

    # Cookie
    COOKIE_SECURE: bool = True      # False в dev
    COOKIE_SAMESITE: str = "lax"
    COOKIE_DOMAIN: str | None = None

    # CORS
    ALLOWED_ORIGINS: list[str] = ["http://localhost:5173"]

    # Qdrant
    QDRANT_URL: str = "http://localhost:6333"
    QDRANT_API_KEY: str | None = None

    # LLM
    OPENAI_API_KEY: str | None = None
    OLLAMA_BASE_URL: str = "http://localhost:11434"
    LLM_PROVIDER: str = "openai"    # "openai" | "ollama"
    LLM_MODEL: str = "gpt-4o"
    EMBEDDING_MODEL: str = "text-embedding-3-small"

    # Storage (для вложений)
    UPLOAD_DIR: str = "/tmp/uploads"

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

app/core/security.py

```
"""
Все операции с паролями и JWT-токенами.
Никаких HTTP-зависимостей — чистые функции.
"""
import hashlib
import secrets
import uuid
from datetime import datetime, timedelta, timezone
from passlib.context import CryptContext
from jose import jwt, JWTError
from app.core.config import get_settings

settings = get_settings()
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto", bcrypt__rounds=12)
```

```
# ── Пароли ──────────────────────────────────────────────────────────────

def hash_password(plain: str) -> str:
    """Возвращает bcrypt-хэш пароля."""
    return pwd_context.hash(plain)

def verify_password(plain: str, hashed: str) -> bool:
    """Сравнивает пароль с хэшем. Устойчиво к timing-атакам."""
    return pwd_context.verify(plain, hashed)

# ── Access Token ────────────────────────────────────────────────────────

def create_access_token(user_id: str, role: str) -> str:
    """
    Payload: {sub: user_id, role: role, type: "access", exp: ...}
    Возвращает подписанный JWT.
    """
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES
    )
    payload = {
        "sub": user_id,
        "role": role,
        "type": "access",
        "exp": expire,
        "iat": datetime.now(timezone.utc),
        "jti": str(uuid.uuid4()),   # уникальный ID токена
    }
    return jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM

def decode_access_token(token: str) -> dict:
    """
    Декодирует и валидирует access_token.
    Raises JWTError если токен невалиден, истёк или неверного типа.
    """
    payload = jwt.decode(token, settings.JWT_SECRET_KEY, algorithms=[settings.JWT_ALGORIT
    if payload.get("type") != "access":
        raise JWTError("Invalid token type")
    return payload

# ── Refresh Token ───────────────────────────────────────────────────────

def generate_refresh_token() -> tuple[str, str]:
    """
    Генерирует raw refresh_token и его SHA-256 хэш для хранения в БД.
    Returns: (raw_token, token_hash)
    raw_token → отправляется клиенту в httpOnly cookie
    token_hash → сохраняется в таблице refresh_tokens
    """
    raw = secrets.token_urlsafe(64)
    token_hash = hashlib.sha256(raw.encode()).hexdigest()
    return raw, token_hash

def hash_refresh_token(raw: str) -> str:
    """Вычисляет хэш raw refresh_token для поиска в БД."""
    return hashlib.sha256(raw.encode()).hexdigest()
```

`app/core/database.py`

```
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine, async_sessionmaker
from sqlalchemy.orm import DeclarativeBase
from app.core.config import get_settings

settings = get_settings()

engine = create_async_engine(
    settings.DATABASE_URL,
    echo=False,          # True для отладки SQL
    pool_size=10,
    max_overflow=20,
)

AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)

class Base(DeclarativeBase):
    pass
```

`app/core/dependencies.py`

```
"""
FastAPI-зависимости: get_db, get_current_user, require_role.
Используются через Depends() в роутерах.
"""
from typing import Annotated
from fastapi import Depends, HTTPException, status, Request
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from sqlalchemy.ext.asyncio import AsyncSession
from jose import JWTError
from app.core.database import AsyncSessionLocal
from app.core.security import decode_access_token
from app.models.user import User, UserRole

bearer = HTTPBearer(auto_error=False)

# ── База данных ──────────────────────────────────────────────────────

async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        yield session

DBSession = Annotated[AsyncSession, Depends(get_db)]

# ── Текущий пользователь ─────────────────────────────────────────────
```

```
async def get_current_user(
    credentials: Annotated[HTTPAuthorizationCredentials | None, Depends(bearer)],
    db: DBSession,
) -> User:
    """
    Читает Bearer-токен из Authorization-заголовка.
    Декодирует JWT, извлекает user_id, загружает пользователя из БД.
    Выбрасывает 401 если токен отсутствует, невалиден или пользователь неактивен.
    """
    if credentials is None:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Not authent
    try:
        payload = decode_access_token(credentials.credentials)
        user_id: str = payload["sub"]
    except (JWTError, KeyError):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid tok

    user = await db.get(User, user_id)
    if user is None or not user.is_active:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="User not fo
    return user

CurrentUser = Annotated[User, Depends(get_current_user)]

# ── Проверка ролей ──────────────────────────────────────────────────────

def require_role(*roles: UserRole):
    """
    Фабрика зависимостей. Пример использования:
        @router.post("/validate")
        async def validate(user: Annotated[User, Depends(require_role(UserRole.ANALYST,
    """
    async def _check(current_user: CurrentUser) -> User:
        if current_user.role not in roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Required roles: {[r.value for r in roles]}"
            )
        return current_user
    return _check

# ── Проверка членства в проекте ─────────────────────────────────────────

async def require_project_member(
    project_id: str,
    current_user: CurrentUser,
    db: DBSession,
) -> User:
    """
    Проверяет, что текущий пользователь является участником проекта.
    ADMIN имеет доступ ко всем проектам без явного членства.
    """
    from app.models.project import ProjectMember
    from sqlalchemy import select
    if current_user.role == UserRole.ADMIN:
        return current_user
```

```
    stmt = select(ProjectMember).where(
        ProjectMember.project_id == project_id,
        ProjectMember.user_id == current_user.id,
    )
    result = await db.execute(stmt)
    if result.scalar_one_or_none() is None:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Not a project
    return current_user
```

5. Backend: SQLAlchemy-модели

app/models/user.py

```
import uuid, enum
from datetime import datetime, timezone
from sqlalchemy import String, Boolean, Enum as SAEnum, func
from sqlalchemy.orm import Mapped, mapped_column
from app.core.database import Base

class UserRole(str, enum.Enum):
    ADMIN = "ADMIN"
    ANALYST = "ANALYST"
    DEVELOPER = "DEVELOPER"
    TESTER = "TESTER"
    MANAGER = "MANAGER"

class User(Base):
    __tablename__ = "users"

    id:           Mapped[str]     = mapped_column(String, primary_key=True, default=la
    email:        Mapped[str]     = mapped_column(String(255), unique=True, nullable=F
    password_hash: Mapped[str]    = mapped_column(String, nullable=False)
    full_name:    Mapped[str]     = mapped_column(String(255), nullable=False)
    role:         Mapped[UserRole] = mapped_column(SAEnum(UserRole), nullable=False, de
    is_active:    Mapped[bool]    = mapped_column(Boolean, nullable=False, default=Tru
    created_at:   Mapped[datetime] = mapped_column(default=func.now())
    updated_at:   Mapped[datetime] = mapped_column(default=func.now(), onupdate=func.no
```

app/models/refresh_token.py

```
import uuid
from datetime import datetime
from sqlalchemy import String, Boolean, ForeignKey, func
from sqlalchemy.orm import Mapped, mapped_column
from app.core.database import Base

class RefreshToken(Base):
    __tablename__ = "refresh_tokens"

    id:         Mapped[str]          = mapped_column(String, primary_key=True, default
    user_id:    Mapped[str]          = mapped_column(String, ForeignKey("users.id", o
```

```
    token_hash:  Mapped[str]          = mapped_column(String, unique=True, nullable=Fals
    family_id:   Mapped[str]          = mapped_column(String, nullable=False)  # UUID-c
    expires_at:  Mapped[datetime]      = mapped_column(nullable=False)
    revoked:     Mapped[bool]         = mapped_column(Boolean, default=False)
    revoked_at:  Mapped[datetime|None] = mapped_column(nullable=True)
    user_agent:  Mapped[str|None]      = mapped_column(String, nullable=True)
    ip_address:  Mapped[str|None]      = mapped_column(String(45), nullable=True)
    created_at:  Mapped[datetime]      = mapped_column(default=func.now())
```

app/models/task.py

```
import uuid
from datetime import datetime
from sqlalchemy import String, Text, ARRAY, Enum as SAEnum, ForeignKey, JSON, func
from sqlalchemy.orm import Mapped, mapped_column
from app.core.database import Base

class TaskStatus(str, enum.Enum):
    DRAFT        = "draft"
    VALIDATING    = "validating"
    NEEDS_REWORK  = "needs_rework"
    READY_FOR_DEV = "ready_for_dev"
    IN_PROGRESS   = "in_progress"
    DONE         = "done"

class Task(Base):
    __tablename__ = "tasks"

    id:               Mapped[str]           = mapped_column(String, primary_key=True, 
    project_id:        Mapped[str]           = mapped_column(String, ForeignKey("projec
    title:            Mapped[str]           = mapped_column(String(500), nullable=False
    content:          Mapped[str]           = mapped_column(Text, default="")
    tags:             Mapped[list[str]]     = mapped_column(ARRAY(String), default=list
    status:           Mapped[TaskStatus]    = mapped_column(SAEnum(TaskStatus), default=
    created_by:        Mapped[str]           = mapped_column(String, ForeignKey("users.
    assigned_to:       Mapped[str|None]      = mapped_column(String, ForeignKey("users.
    validation_result: Mapped[dict|None]     = mapped_column(JSON, nullable=True)
    indexed_at:        Mapped[datetime|None]  = mapped_column(nullable=True)
    created_at:        Mapped[datetime]      = mapped_column(default=func.now())
    updated_at:        Mapped[datetime]      = mapped_column(default=func.now(), onupda
```

6. Backend: Pydantic-схемы

app/schemas/auth.py

```
from pydantic import BaseModel, EmailStr, Field, field_validator

class RegisterRequest(BaseModel):
    email:     EmailStr
    password:  str = Field(min_length=8, max_length=100)
    full_name: str = Field(min_length=2, max_length=255)
```

```
    @field_validator("password")
    @classmethod
    def password_complexity(cls, v: str) -> str:
        if not any(c.isupper() for c in v):
            raise ValueError("Password must contain at least one uppercase letter")
        if not any(c.isdigit() for c in v):
            raise ValueError("Password must contain at least one digit")
        return v

class LoginRequest(BaseModel):
    email:    EmailStr
    password: str

class TokenResponse(BaseModel):
    access_token: str
    token_type:   str = "bearer"
    expires_in:   int         # секунды до истечения access_token

class UserRead(BaseModel):
    id:         str
    email:      str
    full_name:  str
    role:       str
    is_active:  bool
    created_at: str

    model_config = {"from_attributes": True}
```

app/schemas/task.py

```
from pydantic import BaseModel, Field
from typing import Literal

class TaskCreate(BaseModel):
    title:       str = Field(min_length=3, max_length=500)
    content:     str = Field(default="")
    tags:        list[str] = Field(default=[])
    assigned_to: str | None = None

class TaskUpdate(BaseModel):
    title:       str | None = None
    content:     str | None = None
    tags:        list[str] | None = None
    assigned_to: str | None = None

class TaskRead(BaseModel):
    id:               str
    project_id:       str
    title:            str
    content:          str
    tags:             list[str]
    status:           str
    created_by:       str
    assigned_to:      str | None
```

```
    validation_result: dict | None
    created_at:        str
    updated_at:        str
    model_config = {"from_attributes": True}

class ValidationResult(BaseModel):
    verdict:   Literal["approved", "needs_rework"]
    issues:    list[dict]   # [{type, fragment, explanation, recommendation}]
    questions: list[str]
    validated_at: str
```

`app/schemas/message.py`

```
from pydantic import BaseModel

class MessageCreate(BaseModel):
    content: str

class MessageRead(BaseModel):
    id:          str
    task_id:     str
    author_id:    str | None
    agent_name:   str | None
    message_type: str
    content:     str
    source_ref:   dict | None
    created_at:   str
    model_config = {"from_attributes": True}
```

7. Backend: API-роутеры

`app/routers/auth.py` — эндпоинты аутентификации

```
POST   /auth/register       — регистрация нового пользователя
POST   /auth/login          — логин → access_token в JSON + refresh_token в cookie
POST   /auth/refresh        — обновление access_token по refresh_token из cookie
POST   /auth/logout         — ревокация текущего refresh_token
GET    /auth/me             — данные текущего пользователя (требует access_token)
GET    /auth/sessions       — список активных сессий (refresh_tokens) пользователя
DELETE /auth/sessions/{id}  — отозвать конкретную сессию
```

Детальная логика каждого эндпоинта:

`POST /auth/register`

```
Request body: RegisterRequest
Response 201: UserRead

Логика:
```

```
  1. Проверить: пользователь с таким email уже существует? → 409 Conflict
  2. password_hash = hash_password(body.password)
  3. Создать User(email, password_hash, full_name, role=DEVELOPER)
  4. db.add(user); await db.commit()
  5. Вернуть UserRead
```

`POST /auth/login`

```
  Request body: LoginRequest
  Response 200: TokenResponse
  Headers/Cookie: Set-Cookie: refresh_token=<raw>; HttpOnly; Secure; SameSite=Lax; Max-A

  Логика:
  1. Найти user по email → не найден или is_active=False → 401
  2. verify_password(body.password, user.password_hash) → False → 401
  3. access_token = create_access_token(user.id, user.role)
  4. raw_rt, hash_rt = generate_refresh_token()
  5. family_id = str(uuid.uuid4())
  6. Сохранить RefreshToken(user_id, token_hash=hash_rt, family_id, expires_at=now+7d,
                            user_agent=request.headers.get("User-Agent"),
                            ip_address=request.client.host)
  7. Установить httpOnly cookie с raw_rt
  8. Вернуть TokenResponse(access_token, expires_in=900)
```

`POST /auth/refresh`

```
  Cookie: refresh_token=<raw>
  Response 200: TokenResponse (новый access_token)
  Cookie обновляется: новый refresh_token

  Логика:
  1. Прочитать raw_rt из cookie → нет → 401
  2. hash_rt = hash_refresh_token(raw_rt)
  3. Найти RefreshToken по token_hash → не найден → 401
  4. Проверить:
     - revoked == True → ОБНАРУЖЕН REUSE ATTACK:
       * Отозвать ВСЕ токены family_id (UPDATE refresh_tokens SET revoked=True WHERE famil
       * Вернуть 401 "Token reuse detected"
     - expires_at < now() → 401 "Token expired"
  5. Отозвать текущий токен (revoked=True, revoked_at=now())
  6. Создать новый refresh_token (в той же family_id)
  7. Создать новый access_token
  8. Обновить cookie, вернуть TokenResponse
```

`POST /auth/logout`

```
  Cookie: refresh_token=<raw>
  Response 204: No Content

  Логика:
  1. hash_rt из cookie
```

```
  2. Найти и отозвать RefreshToken
  3. Удалить cookie (Set-Cookie с Max-Age=0)
```

`GET /auth/me`

```
  Header: Authorization: Bearer <access_token>
  Response 200: UserRead

  Логика:
  1. get_current_user dependency → User
  2. Вернуть UserRead.from_orm(user)
```

`app/routers/tasks.py` — **задачи**

```
  GET    /projects/{project_id}/tasks             — список задач проекта (с фильтрацией)
  POST   /projects/{project_id}/tasks             — создать задачу (ANALYST, ADMIN)
  GET    /projects/{project_id}/tasks/{task_id}    — получить задачу
  PATCH  /projects/{project_id}/tasks/{task_id}    — обновить задачу (ANALYST, ADMIN)
  DELETE /projects/{project_id}/tasks/{task_id}    — удалить задачу (ANALYST, ADMIN)
  POST   /projects/{project_id}/tasks/{task_id}/attachments  — загрузить вложение

  Query params для GET /tasks:
    status: task_status | None
    tags: list[str] | None    (фильтр по тегам, ANY)
    assigned_to: str | None
    search: str | None        (fulltext по title + content)
    page: int = 1
    size: int = 20
```

`app/routers/validation.py` — **валидация**

```
  POST /tasks/{task_id}/validate

  Request body: {} (пустой, все данные берутся из БД по task_id)
  Response 200: ValidationResult

  Доступ: ANALYST, ADMIN
  Условие: task.status должен быть 'draft' или 'needs_rework'

  Логика:
  1. Загрузить task + attachments из БД
  2. Проверить доступ (пользователь — член проекта задачи)
  3. Обновить task.status = 'validating'
  4. Запустить ValidationGraph (LangGraph): await run_validation_graph(task, db)
  5. Получить результат: {verdict, issues, questions}
  6. Обновить task.status:
     - "approved"    → status = 'ready_for_dev', запустить RAG-индексацию (background task)
     - "needs_rework" → status = 'needs_rework'
```

```
7. Сохранить task.validation_result = {verdict, issues, questions, validated_at}
8. Вернуть ValidationResult
```

`app/routers/chat.py` — **чат**

```
GET  /tasks/{task_id}/messages         — история сообщений (с пагинацией)
POST /tasks/{task_id}/messages         — отправить сообщение

Query params GET:
  before: datetime | None   — сообщения до указанного времени (курсорная пагинация)
  limit: int = 50

POST body: MessageCreate { content: str }
POST response 200: list[MessageRead]    — [сообщение пользователя, ответ агента (если ест

Логика POST:
1. Сохранить сообщение пользователя (author_id = current_user.id, type='general')
2. Запустить ChatGraph: result = await run_chat_graph(task, message, current_user, db)
3. Если агент сформировал ответ → сохранить агентское сообщение (author_id=None, agent_na
4. Вернуть список сообщений (user + agent если есть)
```

`app/routers/projects.py` — **проекты**

```
GET    /projects                       — список проектов пользователя
POST   /projects                       — создать проект (ANALYST, MANAGER, ADMIN)
GET    /projects/{id}                  — детали проекта
PATCH  /projects/{id}                  — обновить (MANAGER, ADMIN)
DELETE /projects/{id}                  — удалить (ADMIN only)

GET    /projects/{id}/members           — список участников
POST   /projects/{id}/members           — добавить участника (MANAGER, ADMIN)
DELETE /projects/{id}/members/{user_id}  — удалить участника

GET    /projects/{id}/rules             — custom rules проекта
POST   /projects/{id}/rules             — добавить правило (ADMIN)
PATCH  /projects/{id}/rules/{rule_id}    — обновить правило
DELETE /projects/{id}/rules/{rule_id}    — удалить правило
```

`app/routers/proposals.py` — **предложения изменений**

```
GET    /tasks/{task_id}/proposals        — список предложений (фильтр по status)
PATCH  /tasks/{task_id}/proposals/{id}   — принять/отклонить (ANALYST, ADMIN)
  body: { status: "accepted" | "rejected" }
```

8. Backend: сервисный слой

`app/services/auth_service.py`

```
"""
Бизнес-логика аутентификации.
Все методы принимают db: AsyncSession, возвращают ORM-объекты или raise HTTPException.
"""
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, update
from datetime import datetime, timedelta, timezone
from fastapi import HTTPException, status, Request, Response
from app.models.user import User
from app.models.refresh_token import RefreshToken
from app.core.security import (hash_password, verify_password,
                               create_access_token, generate_refresh_token,
                               hash_refresh_token)
from app.core.config import get_settings
import uuid

settings = get_settings()

class AuthService:

    @staticmethod
    async def register(email: str, password: str, full_name: str, db: AsyncSession) -> U
        stmt = select(User).where(User.email == email)
        existing = (await db.execute(stmt)).scalar_one_or_none()
        if existing:
            raise HTTPException(status_code=409, detail="Email already registered")
        user = User(email=email, password_hash=hash_password(password), full_name=full_na
        db.add(user)
        await db.commit()
        await db.refresh(user)
        return user

    @staticmethod
    async def login(email: str, password: str, db: AsyncSession,
                    request: Request, response: Response) -> dict:
        stmt = select(User).where(User.email == email)
        user = (await db.execute(stmt)).scalar_one_or_none()
        if not user or not user.is_active or not verify_password(password, user.password
            raise HTTPException(status_code=401, detail="Invalid credentials")

        access_token = create_access_token(user.id, user.role.value)
        raw_rt, hash_rt = generate_refresh_token()
        family_id = str(uuid.uuid4())

        rt = RefreshToken(
            user_id=user.id,
            token_hash=hash_rt,
            family_id=family_id,
            expires_at=datetime.now(timezone.utc) + timedelta(days=settings.REFRESH_TOKE
            user_agent=request.headers.get("User-Agent"),
            ip_address=str(request.client.host) if request.client else None,
```

```
        )
        db.add(rt)
        await db.commit()

        # Устанавливаем httpOnly cookie
        response.set_cookie(
            key="refresh_token",
            value=raw_rt,
            max_age=settings.REFRESH_TOKEN_EXPIRE_DAYS * 86400,
            httponly=True,
            secure=settings.COOKIE_SECURE,
            samesite=settings.COOKIE_SAMESITE,
            domain=settings.COOKIE_DOMAIN,
        )
        return {
            "access_token": access_token,
            "token_type": "bearer",
            "expires_in": settings.ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        }

    @staticmethod
    async def refresh(raw_rt: str, db: AsyncSession,
                      request: Request, response: Response) -> dict:
        token_hash = hash_refresh_token(raw_rt)
        stmt = select(RefreshToken).where(RefreshToken.token_hash == token_hash)
        rt = (await db.execute(stmt)).scalar_one_or_none()

        if rt is None:
            raise HTTPException(status_code=401, detail="Invalid refresh token")

        if rt.revoked:
            # Reuse attack: отзываем всю семью
            await db.execute(
                update(RefreshToken)
                .where(RefreshToken.family_id == rt.family_id)
                .values(revoked=True, revoked_at=datetime.now(timezone.utc))
            )
            await db.commit()
            response.delete_cookie("refresh_token")
            raise HTTPException(status_code=401, detail="Token reuse detected. All sessio

        if rt.expires_at.replace(tzinfo=timezone.utc) < datetime.now(timezone.utc):
            raise HTTPException(status_code=401, detail="Refresh token expired")

        # Отзываем текущий
        rt.revoked = True
        rt.revoked_at = datetime.now(timezone.utc)

        # Создаём новый в той же family
        user = await db.get(User, rt.user_id)
        access_token = create_access_token(user.id, user.role.value)
        new_raw, new_hash = generate_refresh_token()

        new_rt = RefreshToken(
            user_id=user.id,
            token_hash=new_hash,
```

```
            family_id=rt.family_id,
            expires_at=datetime.now(timezone.utc) + timedelta(days=settings.REFRESH_TOKE
            user_agent=request.headers.get("User-Agent"),
            ip_address=str(request.client.host) if request.client else None,
        )
        db.add(new_rt)
        await db.commit()

        response.set_cookie("refresh_token", new_raw,
                            max_age=settings.REFRESH_TOKEN_EXPIRE_DAYS * 86400,
                            httponly=True, secure=settings.COOKIE_SECURE,
                            samesite=settings.COOKIE_SAMESITE)
        return {"access_token": access_token, "token_type": "bearer",
                "expires_in": settings.ACCESS_TOKEN_EXPIRE_MINUTES * 60}
```

9. Агентный слой (LangGraph)

`app/agents/state.py` — TypedDict для всех графов

```
from typing import TypedDict, Literal, Optional

class ValidationState(TypedDict):
    # Входные данные
    task_id:    str
    task_text:  str
    tags:       list[str]
    # Контекст авторизации (передаётся из роутера)
    project_id: str
    user_id:    str
    # Результаты узлов
    ieee_issues:      list[dict]
    custom_violations: list[dict]
    rag_questions:    list[str]
    # Финальный результат
    verdict:    Literal["approved", "needs_rework"] | None
    all_issues: list[dict]

class ChatState(TypedDict):
    task_id:    str
    task_text:  str
    tags:       list[str]
    project_id: str
    user_id:    str
    user_role:  str
    message:    str
    # Результаты маршрутизации
    message_type:   Literal["question", "change_proposal", "general"] | None
    agent_response: str | None
    source_ref:     dict | None
    agent_name:     str | None
    proposal_text:  str | None
    is_duplicate:   bool | None
```

app/agents/validation_graph.py

```
"""
ValidationGraph: StateGraph с 4 последовательными узлами.
Принимает ValidationState, возвращает заполненный ValidationState с verdict.
"""
from langgraph.graph import StateGraph, END
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from app.agents.state import ValidationState
from app.core.config import get_settings

settings = get_settings()

# ── LLM-инициализация ──────────────────────────────────────────────────────
# Вынести в отдельную функцию для поддержки провайдер-независимости
def get_llm():
    if settings.LLM_PROVIDER == "openai":
        return ChatOpenAI(model=settings.LLM_MODEL, temperature=0)
    else:  # ollama
        from langchain_community.chat_models import ChatOllama
        return ChatOllama(model=settings.LLM_MODEL, base_url=settings.OLLAMA_BASE_URL)

# ── Узел 1: IEEE 830 Check ─────────────────────────────────────────────────
async def ieee_check_node(state: ValidationState) -> ValidationState:
    """
    Промпт: проверить task_text на атомарность, однозначность, проверяемость,
    полноту, непротиворечивость по IEEE 830.
    Ожидаемый формат ответа: JSON [{type, fragment, explanation, recommendation}]
    Если нарушений нет — вернуть пустой список.
    """
    llm = get_llm()
    prompt = ChatPromptTemplate.from_messages([
        ("system", """Ты эксперт по инженерии требований. Проверь описание задачи на соот
1. Атомарность (одно требование = одна функция)
2. Однозначность (нет расплывчатых формулировок: "быстро", "удобно", "по возможности")
3. Проверяемость (наличие чётких критериев приёмки)
4. Полнота (охвачены все основные сценарии)
5. Непротиворечивость (нет конфликтующих утверждений)

Верни ТОЛЬКО JSON: [{{"type": str, "fragment": str, "explanation": str, "recommendatio
Если нарушений нет — верни []"""),
        ("human", "Задача:\n{task_text}"),
    ])
    chain = prompt | llm
    result = await chain.ainvoke({"task_text": state["task_text"]})
    # Парсинг JSON из result.content
    import json, re
    match = re.search(r'\[.*\]', result.content, re.DOTALL)
    issues = json.loads(match.group()) if match else []
    return {**state, "ieee_issues": issues}

# ── Узел 2: Custom Rules Check ─────────────────────────────────────────────
async def custom_rules_node(state: ValidationState) -> ValidationState:
    """
    Загружает custom_rules из PostgreSQL для project_id.
```

```
    Фильтрует по tags: правила с applies_to_tags=[] применяются ко всем;
    правила с тегами — только если теги задачи пересекаются с applies_to_tags.
    Передаёт отфильтрованные правила в LLM для проверки.
    """
    from sqlalchemy import select, or_, func
    from app.core.database import AsyncSessionLocal
    from app.models.custom_rule import CustomRule
    async with AsyncSessionLocal() as db:
        stmt = (
            select(CustomRule)
            .where(CustomRule.project_id == state["project_id"])
            .where(CustomRule.is_active == True)
            .where(
                or_(
                    CustomRule.applies_to_tags == [],
                    CustomRule.applies_to_tags.overlap(state["tags"]),
                )
            )
        )
        result = await db.execute(stmt)
        rules = result.scalars().all()

    if not rules:
        return {**state, "custom_violations": []}

    llm = get_llm()
    rules_text = "\n".join([f"- {r.title}: {r.description}" for r in rules])
    prompt = ChatPromptTemplate.from_messages([
        ("system", """Проверь задачу на соответствие пользовательским правилам проекта.
Для каждого нарушенного правила верни JSON:
[{{"rule_title": str, "fragment": str, "recommendation": str}}]
Если нарушений нет — верни [].

Правила проекта:
{rules}"""),
        ("human", "Задача:\n{task_text}"),
    ])
    chain = prompt | llm
    result = await chain.ainvoke({"rules": rules_text, "task_text": state["task_text"]})
    import json, re
    match = re.search(r'\[.*\]', result.content, re.DOTALL)
    violations = json.loads(match.group()) if match else []
    return {**state, "custom_violations": violations}

# ── Узел 3: RAG Questions Check ──────────────────────────────────────────
async def rag_questions_node(state: ValidationState) -> ValidationState:
    """
    Семантический поиск топ-N вопросов из project_questions в Qdrant.
    Фильтрация: payload.tags пересекается с state["tags"].
    Передаёт вопросы в LLM → LLM дополнительно проверяет задачу по этим вопросам.
    """
    from langchain_qdrant import QdrantVectorStore
    from langchain_openai import OpenAIEmbeddings
    from qdrant_client import QdrantClient

    client = QdrantClient(url=settings.QDRANT_URL, api_key=settings.QDRANT_API_KEY)
```

```
    embeddings = OpenAIEmbeddings(model=settings.EMBEDDING_MODEL)
    store = QdrantVectorStore(client=client, collection_name="project_questions", embed

    from qdrant_client.models import Filter, FieldCondition, MatchAny
    qdrant_filter = None
    if state["tags"]:
        qdrant_filter = Filter(
            must=[FieldCondition(key="tags", match=MatchAny(any=state["tags"]))]
        )
    docs = await store.asimilarity_search(
        state["task_text"], k=5, filter=qdrant_filter
    )
    if not docs:
        return {**state, "rag_questions": []}

    questions = [d.page_content for d in docs]
    llm = get_llm()
    prompt = ChatPromptTemplate.from_messages([
        ("system", """На основе типичных вопросов к задачам проекта проверь, ответила ли
Верни список вопросов, на которые в задаче НЕТ явного ответа (строки).
Верни ТОЛЬКО JSON: ["вопрос 1", "вопрос 2"] или []"""),
        ("human", "Задача:\n{task_text}\n\nТипичные вопросы:\n{questions}"),
    ])
    chain = prompt | llm
    result = await chain.ainvoke({
        "task_text": state["task_text"],
        "questions": "\n".join(f"- {q}" for q in questions),
    })
    import json, re
    match = re.search(r'\[.*\]', result.content, re.DOTALL)
    unanswered = json.loads(match.group()) if match else []
    return {**state, "rag_questions": unanswered}

# ── Узел 4: Aggregator ──────────────────────────────────────────────────
async def aggregator_node(state: ValidationState) -> ValidationState:
    all_issues = state["ieee_issues"] + state["custom_violations"]
    # Вопросы без ответа не являются дефектами сами по себе, но влияют на вердикт
    has_critical = len(all_issues) > 0 or len(state["rag_questions"]) > 2
    verdict = "needs_rework" if has_critical else "approved"
    return {**state, "verdict": verdict, "all_issues": all_issues}

# ── Сборка графа ────────────────────────────────────────────────────────
def build_validation_graph():
    graph = StateGraph(ValidationState)
    graph.add_node("ieee_check", ieee_check_node)
    graph.add_node("custom_rules", custom_rules_node)
    graph.add_node("rag_questions", rag_questions_node)
    graph.add_node("aggregator", aggregator_node)
    graph.set_entry_point("ieee_check")
    graph.add_edge("ieee_check", "custom_rules")
    graph.add_edge("custom_rules", "rag_questions")
    graph.add_edge("rag_questions", "aggregator")
    graph.add_edge("aggregator", END)
    return graph.compile()

validation_graph = build_validation_graph()
```

```
# ── Публичная функция ──────────────────────────────────────────────────────
async def run_validation_graph(task, db) -> dict:
    initial_state: ValidationState = {
        "task_id":    str(task.id),
        "task_text":  f"{task.title}\n\n{task.content}",
        "tags":       task.tags or [],
        "project_id": str(task.project_id),
        "user_id":    str(task.created_by),
        "ieee_issues": [], "custom_violations": [], "rag_questions": [],
        "verdict": None, "all_issues": [],
    }
    result = await validation_graph.ainvoke(initial_state)
    return result
```

app/agents/chat_graph.py

```
"""
ChatGraph: Supervisor Pattern.
Manager Node классифицирует сообщение → routing → QA Agent | ChangeTracker | (нет ответа)
"""
from langgraph.graph import StateGraph, END
from langchain_core.prompts import ChatPromptTemplate
from app.agents.state import ChatState

async def manager_node(state: ChatState) -> ChatState:
    """
    Классифицирует сообщение. Результат: state["message_type"]
    Классы: "question" | "change_proposal" | "general"
    """
    from app.agents.validation_graph import get_llm
    llm = get_llm()
    prompt = ChatPromptTemplate.from_messages([
        ("system", """Классифицируй сообщение участника команды по одному из трёх типов:
- "question": вопрос по требованиям, контексту задачи, поведению системы
- "change_proposal": предложение изменить формулировку, критерии приёмки, разделить задач
- "general": организационное обсуждение, статусы, приветствия (не требует ответа агента)

Верни ТОЛЬКО JSON: {{"type": "...", "extracted_proposal": "..." (только для change_prop
Контекст задачи: {task_text}
Роль автора: {user_role}"""),
        ("human", "{message}"),
    ])
    chain = prompt | llm
    result = await chain.ainvoke({
        "task_text": state["task_text"][:500],  # укороченный контекст для экономии токено
        "user_role": state["user_role"],
        "message": state["message"],
    })
    import json, re
    match = re.search(r'\{.*\}', result.content, re.DOTALL)
    data = json.loads(match.group()) if match else {"type": "general"}
    return {
        **state,
        "message_type": data.get("type", "general"),
```

```
        "proposal_text": data.get("extracted_proposal"),
    }

def router(state: ChatState) -> str:
    """Условный edge: определяет следующий узел по message_type."""
    return state.get("message_type", "general")

async def qa_agent_node(state: ChatState) -> ChatState:
    """
    1. Ищет ответ в тексте задачи (LLM)
    2. Если не найден → RAG-поиск в task_knowledge (Qdrant)
    3. Если всё равно нет → сохраняет вопрос в project_questions
    Возвращает agent_response с указанием источника (≤300 слов).
    """
    from app.agents.validation_graph import get_llm
    from langchain_qdrant import QdrantVectorStore
    from langchain_openai import OpenAIEmbeddings
    from qdrant_client import QdrantClient
    from qdrant_client.models import Filter, FieldCondition, MatchValue
    from app.core.config import get_settings
    settings = get_settings()
    llm = get_llm()

    # Шаг 1: поиск в тексте задачи
    step1_prompt = ChatPromptTemplate.from_messages([
        ("system", "Найди ответ на вопрос пользователя в тексте задачи. "
                   "Если ответ есть — дай его (≤300 слов) и укажи конкретную цитату. "
                   "Если ответа нет — верни ТОЛЬКО строку 'NOT_FOUND'."),
        ("human", "Задача: {task_text}\n\nВопрос: {message}"),
    ])
    chain1 = step1_prompt | llm
    r1 = await chain1.ainvoke({"task_text": state["task_text"], "message": state["message

    if "NOT_FOUND" not in r1.content:
        return {**state, "agent_response": r1.content, "agent_name": "QAAgent",
                "source_ref": {"type": "task_text", "task_id": state["task_id"]}}

    # Шаг 2: RAG-поиск
    client = QdrantClient(url=settings.QDRANT_URL, api_key=settings.QDRANT_API_KEY)
    embeddings = OpenAIEmbeddings(model=settings.EMBEDDING_MODEL)
    store = QdrantVectorStore(client=client, collection_name="task_knowledge", embeddin
    qdrant_filter = Filter(
        must=[FieldCondition(key="task_id", match=MatchValue(value=state["task_id"]))]
    ) if state["tags"] else None
    docs = await store.asimilarity_search(state["message"], k=4, filter=qdrant_filter)

    if docs:
        context = "\n\n".join(d.page_content for d in docs)
        step2_prompt = ChatPromptTemplate.from_messages([
            ("system", "Ответь на вопрос, используя контекст из базы знаний. "
                       "Ответ ≤300 слов. Укажи, из какого источника взята информация."),
            ("human", "Контекст:\n{context}\n\nВопрос: {message}"),
        ])
        r2 = await (step2_prompt | llm).ainvoke({"context": context, "message": state["m
        chunk_ids = [d.metadata.get("chunk_id") for d in docs if d.metadata.get("chunk_i
        return {**state, "agent_response": r2.content, "agent_name": "QAAgent",
```

```
                "source_ref": {"type": "rag", "collection": "task_knowledge", "chunk_ids

    # Шаг 3: ответа нет → сохраняем вопрос в project_questions
    reformulate_prompt = ChatPromptTemplate.from_messages([
        ("system", "Переформулируй вопрос пользователя как чёткий, канонический вопрос "
                   "для базы знаний проекта. Верни ТОЛЬКО текст вопроса."),
        ("human", "{message}"),
    ])
    r3 = await (reformulate_prompt | llm).ainvoke({"message": state["message"]})
    canonical_q = r3.content.strip()

    # Сохранить в Qdrant project_questions
    from langchain_core.documents import Document
    doc = Document(page_content=canonical_q,
                   metadata={"tags": state["tags"], "task_id": state["task_id"],
                             "user_role": state["user_role"]})
    await store.aadd_documents([doc])  # использовать коллекцию project_questions

    return {**state, "agent_response": "Ответ не найден в базе знаний. Вопрос сохранён дл
            "agent_name": "QAAgent", "source_ref": {"type": "no_answer", "saved_questio

async def change_tracker_node(state: ChatState) -> ChatState:
    """
    1. Дедупликация через Qdrant task_proposals
    2. Если новое → сохранить в change_proposals (PostgreSQL) + task_proposals (Qdrant)
    """
    from app.agents.validation_graph import get_llm
    from app.core.config import get_settings
    from qdrant_client import QdrantClient
    from langchain_qdrant import QdrantVectorStore
    from langchain_openai import OpenAIEmbeddings
    settings = get_settings()

    proposal = state.get("proposal_text") or state["message"]

    client = QdrantClient(url=settings.QDRANT_URL, api_key=settings.QDRANT_API_KEY)
    embeddings = OpenAIEmbeddings(model=settings.EMBEDDING_MODEL)
    store = QdrantVectorStore(client=client, collection_name="task_proposals", embeddin

    similar = await store.asimilarity_search_with_score(proposal, k=3)
    is_duplicate = any(score > 0.92 for _, score in similar)

    if is_duplicate:
        top_match = similar[0][0]
        return {**state, "agent_response": f"Аналогичное предложение уже существует в баз
                "agent_name": "ChangeTrackerAgent", "is_duplicate": True}

    # Сохранение (вызывается через сервис, чтобы не смешивать DB-логику с агентом)
    # Здесь возвращаем сигнал — роутер/сервис выполняет INSERT
    return {**state, "agent_response": "Предложение изменения сохранено и передано аналит
            "agent_name": "ChangeTrackerAgent", "is_duplicate": False,
            "proposal_text": proposal}

async def general_node(state: ChatState) -> ChatState:
    """Обычное обсуждение — агент не отвечает."""
    return {**state, "agent_response": None, "agent_name": None}
```

```
def build_chat_graph():
    graph = StateGraph(ChatState)
    graph.add_node("manager", manager_node)
    graph.add_node("qa_agent", qa_agent_node)
    graph.add_node("change_tracker", change_tracker_node)
    graph.add_node("general", general_node)
    graph.set_entry_point("manager")
    graph.add_conditional_edges("manager", router, {
        "question":        "qa_agent",
        "change_proposal": "change_tracker",
        "general":         "general",
    })
    graph.add_edge("qa_agent", END)
    graph.add_edge("change_tracker", END)
    graph.add_edge("general", END)
    return graph.compile()

chat_graph = build_chat_graph()
```

10. Frontend: файловая структура

```
frontend/
├── index.html
├── package.json         # vite, react, typescript, axios, react-router-dom, zustand
├── tsconfig.json
├── vite.config.ts
│
└── src/
    ├── main.tsx                # точка входа
    ├── App.tsx                 # роутер + AuthProvider
    │
    ├── api/
    │   ├── client.ts           # axios instance с interceptors
    │   ├── authApi.ts          # login, register, refresh, logout, me
    │   ├── tasksApi.ts         # CRUD задач, validate, attachments
    │   ├── chatApi.ts          # messages GET/POST
    │   ├── projectsApi.ts       # CRUD проектов, members, rules
    │   └── proposalsApi.ts      # GET/PATCH proposals
    │
    ├── store/
    │   ├── authStore.ts         # Zustand: user, accessToken, setToken, logout
    │   └── uiStore.ts           # Zustand: loading, notifications
    │
    ├── auth/
    │   ├── AuthProvider.tsx     # инициализация при загрузке страницы
    │   ├── useAuth.ts           # хук доступа к authStore
    │   ├── ProtectedRoute.tsx    # HOC: редирект на /login если нет токена
    │   ├── RoleGuard.tsx        # HOC: редирект если роль не подходит
    │   └── pages/
    │       ├── LoginPage.tsx
    │       └── RegisterPage.tsx
    │
    ├── features/
```

```
    │   ├── projects/
    │   │   ├── ProjectList.tsx
    │   │   ├── ProjectCard.tsx
    │   │   └── CreateProjectModal.tsx
    │   │
    │   ├── tasks/
    │   │   ├── TaskList.tsx
    │   │   ├── TaskCard.tsx
    │   │   ├── TaskDetailPage.tsx
    │   │   ├── TaskForm.tsx           # создание / редактирование
    │   │   ├── ValidationPanel.tsx     # показывает результаты валидации
    │   │   └── AttachmentUpload.tsx
    │   │
    │   ├── chat/
    │   │   ├── ChatWindow.tsx
    │   │   ├── MessageList.tsx
    │   │   ├── MessageBubble.tsx       # разный вид для user / agent
    │   │   └── MessageInput.tsx
    │   │
    │   └── admin/
    │       ├── UserList.tsx
    │       └── CustomRulesEditor.tsx
    │
    └── shared/
        ├── components/
        │   ├── Layout.tsx             # AppBar + Sidebar + Outlet
        │   ├── LoadingSpinner.tsx
        │   ├── ErrorBoundary.tsx
        │   └── ConfirmDialog.tsx
        └── hooks/
            ├── useApi.ts              # обёртка над fetch с loading/error state
            └── useDebounce.ts
```

11. Frontend: API-клиент

`src/api/client.ts`

```
import axios, { AxiosInstance, InternalAxiosRequestConfig } from "axios";
import { useAuthStore } from "@/store/authStore";

const API_BASE = import.meta.env.VITE_API_URL ?? "http://localhost:8000";

// Флаг для предотвращения параллельных refresh-запросов
let isRefreshing = false;
let refreshQueue: Array<(token: string) => void> = [];

export const apiClient: AxiosInstance = axios.create({
  baseURL: API_BASE,
  withCredentials: true,   // ВАЖНО: отправлять httpOnly cookie с refresh_token
  headers: { "Content-Type": "application/json" },
});

// ── Request Interceptor: добавить access_token ──────────────────────────
```

```
apiClient.interceptors.request.use((config: InternalAxiosRequestConfig) => {
  const token = useAuthStore.getState().accessToken;
  if (token && config.headers) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// ── Response Interceptor: обработка 401 и авто-refresh ──────────────────
apiClient.interceptors.response.use(
  (response) => response,
  async (error) => {
    const original = error.config;

    // Если 401 и запрос ещё не ретраился и это не /auth/* эндпоинт
    if (
      error.response?.status === 401 &&
      !original._retry &&
      !original.url?.includes("/auth/")
    ) {
      original._retry = true;

      if (isRefreshing) {
        // Ставим запрос в очередь — дождёмся нового токена
        return new Promise((resolve) => {
          refreshQueue.push((token: string) => {
            original.headers.Authorization = `Bearer ${token}`;
            resolve(apiClient(original));
          });
        });
      }

      isRefreshing = true;
      try {
        // POST /auth/refresh — refresh_token едет в httpOnly cookie автоматически
        const { data } = await apiClient.post<{ access_token: string }>(
          "/auth/refresh"
        );
        const newToken = data.access_token;
        useAuthStore.getState().setAccessToken(newToken);

        // Разблокировать очередь
        refreshQueue.forEach((cb) => cb(newToken));
        refreshQueue = [];

        original.headers.Authorization = `Bearer ${newToken}`;
        return apiClient(original);
      } catch {
        // Refresh не удался — разлогинить
        useAuthStore.getState().logout();
        window.location.href = "/login";
        return Promise.reject(error);
      } finally {
        isRefreshing = false;
      }
    }
```

```
    return Promise.reject(error);
  }
);
```

`src/api/authApi.ts`

```
import { apiClient } from "./client";

export interface TokenResponse {
  access_token: string;
  token_type: string;
  expires_in: number;
}

export interface UserRead {
  id: string;
  email: string;
  full_name: string;
  role: "ADMIN" | "ANALYST" | "DEVELOPER" | "TESTER" | "MANAGER";
  is_active: boolean;
  created_at: string;
}

export const authApi = {
  login: (email: string, password: string) =>
    apiClient.post<TokenResponse>("/auth/login", { email, password }),

  register: (email: string, password: string, full_name: string) =>
    apiClient.post<UserRead>("/auth/register", { email, password, full_name }),

  refresh: () =>
    apiClient.post<TokenResponse>("/auth/refresh"),

  logout: () =>
    apiClient.post<void>("/auth/logout"),

  me: () =>
    apiClient.get<UserRead>("/auth/me"),
};
```

12. Frontend: AuthContext

`src/store/authStore.ts`

```
import { create } from "zustand";

interface User {
  id: string;
  email: string;
  full_name: string;
  role: string;
```

```
}

interface AuthState {
  user: User | null;
  accessToken: string | null;       // хранится ТОЛЬКО в памяти (не localStorage)
  isInitialized: boolean;           // завершена ли проверка токена при старте
  setUser: (user: User) => void;
  setAccessToken: (token: string) => void;
  logout: () => void;
  setInitialized: () => void;
}

export const useAuthStore = create<AuthState>((set) => ({
  user: null,
  accessToken: null,
  isInitialized: false,

  setUser: (user) => set({ user }),
  setAccessToken: (token) => set({ accessToken: token }),
  setInitialized: () => set({ isInitialized: true }),

  logout: () => set({ user: null, accessToken: null }),
}));
```

`src/auth/AuthProvider.tsx`

```
import { useEffect } from "react";
import { authApi } from "@/api/authApi";
import { useAuthStore } from "@/store/authStore";

/**
 * При монтировании пытается обновить access_token через refresh_token (из cookie).
 * Это позволяет пережить перезагрузку страницы — пользователь остаётся залогиненным.
 * Должен оборачивать всё приложение ДО роутера.
 */
export function AuthProvider({ children }: { children: React.ReactNode }) {
  const { setAccessToken, setUser, logout, setInitialized } = useAuthStore();

  useEffect(() => {
    const init = async () => {
      try {
        // Пытаемся обновить токен (refresh_token в httpOnly cookie)
        const { data: tokenData } = await authApi.refresh();
        setAccessToken(tokenData.access_token);
        // Загружаем данные пользователя
        const { data: userData } = await authApi.me();
        setUser(userData);
      } catch {
        // Нет валидного refresh_token — пользователь не авторизован
        logout();
      } finally {
        setInitialized();
      }
    };
    init();
```

```
  }, []);

  return <>{children}</>;
}
```

13. Frontend: роутинг

`src/App.tsx`

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider } from "@/auth/AuthProvider";
import { ProtectedRoute } from "@/auth/ProtectedRoute";
import { RoleGuard } from "@/auth/RoleGuard";
import LoginPage from "@/auth/pages/LoginPage";
import RegisterPage from "@/auth/pages/RegisterPage";
import { Layout } from "@/shared/components/Layout";
import ProjectList from "@/features/projects/ProjectList";
import TaskDetailPage from "@/features/tasks/TaskDetailPage";
import TaskList from "@/features/tasks/TaskList";
import UserList from "@/features/admin/UserList";
import CustomRulesEditor from "@/features/admin/CustomRulesEditor";

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          {/* Публичные маршруты */}
          <Route path="/login" element={<LoginPage />} />
          <Route path="/register" element={<RegisterPage />} />

          {/* Защищённые маршруты (требует любой авторизации) */}
          <Route element={<ProtectedRoute />}>
            <Route element={<Layout />}>
              <Route path="/" element={<Navigate to="/projects" replace />} />
              <Route path="/projects" element={<ProjectList />} />
              <Route path="/projects/:projectId/tasks" element={<TaskList />} />
              <Route path="/projects/:projectId/tasks/:taskId" element={<TaskDetailPage /

              {/* Только ADMIN */}
              <Route element={<RoleGuard allowedRoles={["ADMIN"]} />}>
                <Route path="/admin/users" element={<UserList />} />
                <Route path="/admin/projects/:projectId/rules" element={<CustomRulesEdit
              </Route>
            </Route>
          </Route>

          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
```

```
  );
}
```

`src/auth/ProtectedRoute.tsx`

```
import { Navigate, Outlet } from "react-router-dom";
import { useAuthStore } from "@/store/authStore";
import { LoadingSpinner } from "@/shared/components/LoadingSpinner";

export function ProtectedRoute() {
  const { user, isInitialized } = useAuthStore();

  // Ждём завершения инициализации (проверки refresh_token)
  if (!isInitialized) return <LoadingSpinner fullscreen />;

  // Не авторизован — редирект на /login
  if (!user) return <Navigate to="/login" replace />;

  return <Outlet />;
}
```

`src/auth/RoleGuard.tsx`

```
import { Navigate, Outlet } from "react-router-dom";
import { useAuthStore } from "@/store/authStore";

interface Props {
  allowedRoles: string[];
}

export function RoleGuard({ allowedRoles }: Props) {
  const { user } = useAuthStore();

  if (!user || !allowedRoles.includes(user.role)) {
    return <Navigate to="/" replace />;
  }
  return <Outlet />;
}
```

14. Frontend: компоненты

`src/auth/pages/LoginPage.tsx`

```
/**
 * СОСТОЯНИЕ: email (string), password (string), error (string|null), isLoading (bool)
 *
 * ПОВЕДЕНИЕ:
 * 1. Форма с полями email + password
 * 2. При submit:
```

```
 *    - setLoading(true)
 *    - POST /auth/login через authApi.login(email, password)
 *    - При успехе: setAccessToken(data.access_token), загрузить /auth/me,
 *      setUser(user), navigate("/projects")
 *    - При ошибке 401: setError("Неверный email или пароль")
 *    - setLoading(false)
 * 3. Ссылка "Нет аккаунта? Зарегистрироваться" → /register
 *
 * ВАЛИДАЦИЯ:
 * - email: не пустой, формат email
 * - password: не пустой
 *
 * UI: карточка по центру страницы, без Layout/Sidebar
 */
```

src/features/tasks/TaskDetailPage.tsx

```
/**
 * СОСТОЯНИЕ:
 * - task: TaskRead | null
 * - messages: MessageRead[]
 * - isLoadingTask: bool
 * - isSendingMessage: bool
 * - validationLoading: bool
 *
 * LAYOUT:
 * - Левая колонка (60%): карточка задачи (title, content, tags, status, validation_resul
 * - Правая колонка (40%): ChatWindow с историей сообщений
 *
 * ПОВЕДЕНИЕ:
 * 1. При монтировании: GET /projects/{projectId}/tasks/{taskId}
 *    + GET /tasks/{taskId}/messages?limit=50
 * 2. Кнопка "Отправить на валидацию":
 *    - Видна только ANALYST/ADMIN
 *    - Активна только когда task.status in ['draft', 'needs_rework']
 *    - Клик → POST /tasks/{taskId}/validate → обновить task.status, task.validation_resul
 *    - Показать ValidationPanel с результатом
 * 3. ChatWindow:
 *    - Отображает MessageBubble для каждого сообщения
 *    - MessageBubble для agent_answer/agent_proposal — другой цвет + иконка агента
 *    - Отправка сообщения: POST /tasks/{taskId}/messages
 *      → добавить user message + agent message (если есть) в список
 * 4. Редактирование задачи: PATCH (только ANALYST/ADMIN, только status='draft')
 */
```

src/features/chat/MessageBubble.tsx

```
/**
 * Props: message: MessageRead
 *
 * ЛОГИКА ОТОБРАЖЕНИЯ:
 * - author_id != null → обычное сообщение пользователя:
 *     выравнивание по правому краю, серый фон
```

```
 * - agent_name = 'QAAgent' → фон #E8F4F8, иконка □, шапка "QA Агент"
 * - agent_name = 'ChangeTrackerAgent' → фон #FEF9E7, иконка □, шапка "Трекер изменений"
 * - message_type = 'agent_answer' и source_ref присутствует:
 *     показать блок "Источник:" с ссылкой на задачу или чанк
 *
 * Метка времени: created_at в формате HH:mm (дата если не сегодня)
 */
```

15. Переменные окружения

`backend/.env`

```
# Database
DATABASE_URL=postgresql+asyncpg://app_user:app_pass@localhost:5432/taskplatform

# JWT (генерировать: python -c "import secrets; print(secrets.token_urlsafe(64))")
JWT_SECRET_KEY=your_64_byte_secret_here
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7

# Cookie
COOKIE_SECURE=false        # true в production (HTTPS)
COOKIE_SAMESITE=lax
COOKIE_DOMAIN=             # пусто для localhost

# CORS
ALLOWED_ORIGINS=["http://localhost:5173"]

# Qdrant
QDRANT_URL=http://localhost:6333
QDRANT_API_KEY=

# LLM
LLM_PROVIDER=openai        # openai | ollama
OPENAI_API_KEY=sk-...
LLM_MODEL=gpt-4o
EMBEDDING_MODEL=text-embedding-3-small

# Storage
UPLOAD_DIR=/tmp/uploads
```

`frontend/.env`

```
VITE_API_URL=http://localhost:8000
```

16. Docker Compose

```yaml
version: "3.9"

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: app_pass
      POSTGRES_DB: taskplatform
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app_user"]
      interval: 5s
      timeout: 5s
      retries: 5

  qdrant:
    image: qdrant/qdrant:latest
    ports:
      - "6333:6333"
      - "6334:6334"
    volumes:
      - qdrant_data:/qdrant/storage

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    env_file: ./backend/.env
    depends_on:
      postgres:
        condition: service_healthy
      qdrant:
        condition: service_started
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --reload

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "5173:5173"
    env_file: ./frontend/.env
    depends_on:
      - backend

volumes:
```

```
  pgdata:
  qdrant_data:
```

Резюме для ИИ-разработчика

Что реализовать в первую очередь (MVP)

Приоритет	Задача	Файлы
P0	БД: создать все таблицы	DDL из раздела 2
P0	Backend: core (config, security, database, dependencies)	`app/core/*`
P0	Backend: роутер `/auth/*` + AuthService	`routers/auth.py`, `services/auth_service.py`
P0	Frontend: AuthProvider + LoginPage + ProtectedRoute	`auth/*`
P0	Frontend: api/client.ts с interceptors	`api/client.ts`
P1	Backend: CRUD задач, проектов	`routers/tasks.py`, `routers/projects.py`
P1	Frontend: TaskList, TaskDetailPage, ChatWindow	`features/*`
P2	LangGraph: ValidationGraph	`agents/validation_graph.py`
P2	LangGraph: ChatGraph	`agents/chat_graph.py`
P3	LangGraph: RAG Indexing Pipeline	`agents/rag_pipeline.py`
P3	Qdrant: инициализация коллекций при старте	`main.py` startup event

Зависимости (pyproject.toml / package.json)

Python:

```
fastapi>=0.115, uvicorn[standard], sqlalchemy[asyncio]>=2.0,
asyncpg, pydantic>=2.0, pydantic-settings, python-jose[cryptography],
passlib[bcrypt], python-multipart, langchain>=0.3, langgraph>=0.2,
langchain-openai, langchain-qdrant, langchain-community,
qdrant-client, httpx, alembic
```

Node.js:

```
react, react-dom, react-router-dom@6, axios, zustand,
typescript, vite, @types/react, tailwindcss
```